- It gives you flexible ways of invoking the tests.

## Assertions == Expected Results

Rather than have you write series of individual if or unless statements in your tests, the testing framework allows you to define assertions that achieve the same thing. Although a number of different styles of assertion exist, they all follow basically the same pattern. Each gives you a way of specifying an expected result and a way of passing in the actual outcome. If the actual value doesn't match the expected value, the assertion outputs a nice message and records the failure.

For example, we could rewrite our previous test of the Roman class using minitest. For now, ignore the scaffolding code at the start and end, and just look at the assert_equal method:

unittesting/test_romanbug1.rb

```
require_relative "romanbug"
require "minitest/autorun"

class TestRoman < Minitest::Test
  def test_simple
    assert_equal("i", Roman.new(1).to_s)
    assert_equal("ix", Roman.new(9).to_s)
  end
end
```

*produces:*

```
Run options: --seed 38457
# Running:

.
Finished in 0.000382s, 2617.8011 runs/s, 5235.6021 assertions/s.
1 runs, 2 assertions, 0 failures, 0 errors, 0 skips
```

The first assertion says that we're expecting the Roman number string representation of 1 to be "i," and the second test says we expect 9 to be "ix."

We can run the test by just running the file as a Ruby file—the minitest/autorun module will automatically load and run our tests, more on that in a bit. Luckily for us, both expectations are met, and the tracing reports that our tests pass.

Let's add a few more tests:

unittesting/test_romanbug2.rb

```
require_relative "romanbug"
require "minitest/autorun"

class TestRoman < Minitest::Test
  def test_simple
    assert_equal("i", Roman.new(1).to_s)
    assert_equal("ii", Roman.new(2).to_s)
    assert_equal("iii", Roman.new(3).to_s)
    assert_equal("iv", Roman.new(4).to_s)
    assert_equal("ix", Roman.new(9).to_s)
  end
end
```

*produces:*

```
Run options: --seed 45617
```